Assignment 5: MPI and OpenMP Parallelization

Matthew Lochridge

March 2026

1 Serial

We start with the provided serial C++ code to solve the N-body problem. A Bash script was written to compile and iteratively run nbody.cc with increasing N and record runtime for each N. Fig. [1] depicts the serial runtime scaling for N up to 16,384 masses.

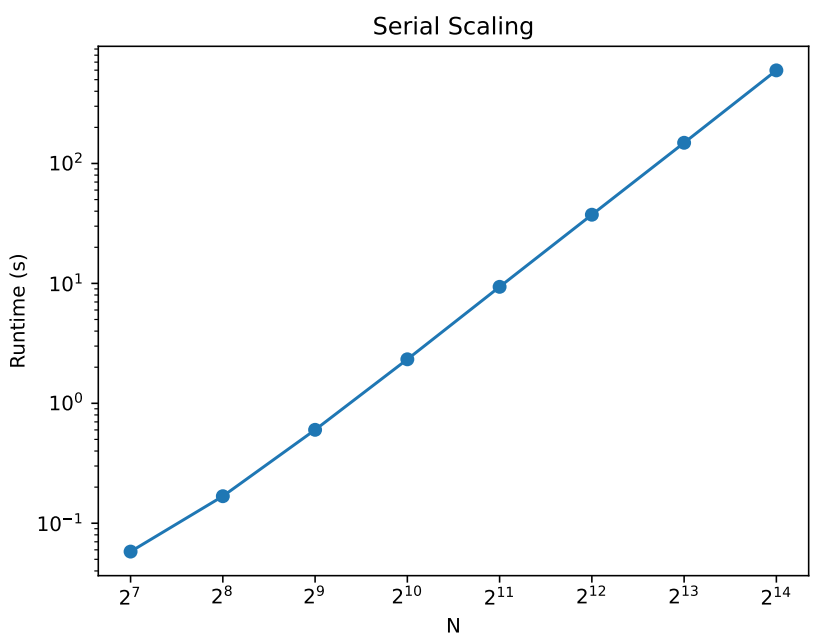


Figure 1: Serial runtime versus N (log scale).

2 Shared-Memory OpenMP

A copy of the serial code was modified to use OpenMP for shared-memory parallelism. Fig. [2] shows key code snippets. The outermost for loops in the initial_conditions, acceleration, and timestep functions, as well as the kinetic energy calculation in the main function, were made parallel with #pragma omp parallel for.

Since the Mersenne twister engine and uniform real distribution standard classes have complex constructors, they cannot directly be made thread-private. This was resolved by defining global pointers for the gen and ran variables, making those pointers thread-private, initializing the RNGs in a #pragma omp parallel block just after the timer starts, and similarly releasing the memory just after the kinetic energy calculations.

A Bash script was written to compile and iteratively run nbody_omp.cc using an interactive session on a Ganymede2 node, with $N = 16,384$ and n_{threads} increasing up to 128. Fig. [8] depicts the strong scaling behavior, and Fig. [3] shows the Ganymede2 terminal log.

3 Distributed-Memory MPI

Another copy of the serial code was modified to use MPI for distributed-memory parallelism. Key code snippets are shown in Fig. [4]. Additional global variables and the setup_parallelism() function were added as specified in the instructions. The ranges of the outermost for loops in the initial_conditions, acceleration, and timestep functions, as well as the kinetic energy calculation in the main function, were adjusted to cover the local ranges for each rank. The positions and velocities at each timestep were gathered by all ranks, and the kinetic energies were reduced to the root rank.

The provided arguments for MPI_Allgatherv caused fatal errors on Ganymede2 due to buffer aliasing. Since the send buffers are subsets of the corresponding receive buffers, the send buffers, send counts, and send types were replaced with MPI_IN_PLACE, 0, and MPI_DATATYPE_NULL, respectively, which resolved the buffer aliasing error.

As for the OpenMP method, a Bash script was written to compile and iteratively run nbody_mpi.cc using an interactive session on a Ganymede2 node, with $N = 16,384$ and n_{ranks} increasing up to 128. Fig. [8] depicts the strong scaling behavior alongside OpenMP, and Fig. [5] shows the Ganymede2 terminal log.

4 Shared-Memory MPI

A copy of the distributed-memory MPI code was modified to utilize shared memory instead. Key code snippets are shown in Fig [6].

As for the previous parallel methods, a Bash script was written to compile and iteratively run nbody_mpi_shared.cc using an interactive session on a

Ganymede2 node, with $N = 16,384$ and n_{ranks} increasing up to 128. Fig. [8] depicts the strong scaling behavior alongside the previous parallel methods, and Fig. [7] shows the Ganymede2 terminal log.

5 Scaling Discussion

As shown in Fig. [8], the OpenMP, distributed-memory MPI, and shared-memory MPI methods scale similarly for low parallelization, diverging from one another around 32 parallel processors. Consistently, the OpenMP method scaled better for high parallelization, while runtime for the distributed-memory MPI method appeared to converge roughly toward ∼30 seconds, and runtime for the shared-memory MPI method spiked at 32 ranks before speeding up again.

For most of the runs, shared-memory MPI performed consistently worse than the other methods, presumably due to the added overhead from accessing shared memory and barriers to avoid race conditions. Around 128 ranks, shared-memory MPI begins outperforming distributed-memory MPI, since the number of data copies in the distributed-memory version becomes large. Consistently, OpenMP outperforms MPI at this scale, since it is built for shared-memory by default, and these tests were carried out on only one HPC node.

6 Local Scaling Validation

As a sanity check, the same parallel codes were run on my laptop to ensure correctness and reproducibility. Each previous method was tested with $N = 16,384$ and up to 16 parallel processors (i.e. the number of cores in my laptop). The results are shown in Fig. [9].

7 Hybrid MPI + OpenMP

Finally, another copy of the distributed-memory MPI code was modified to additionally utilize OpenMP for shared-memory threading within each rank. Key code snippets are shown in Fig. [10]. This amounted to adding the same OpenMP pragma statements to the distributed-memory MPI code. The same quirks as previously encountered (the complex construction of the std::mt19937 and std::uniform_real_distribution classes, and the overlapping send and receive buffers in the MPI_Allgatherv calls) were resolved in the same way as before.

A Slurm script was written to compile and run nbody_mpi_omp.cc once on Ganymede2 using 2 nodes for a specified input N, n_{ranks}, and n_{threads}, with a time limit of 15 minutes. The largest problem completed within that time limit used $N = 196,608$, $n_{\text{ranks}} = 4$, and $n_{\text{threads}} = 32$. This final run finished in about 13.5 minutes, as shown in Fig. [11].

```cpp
// Random number generator
static std::mt19937* gen_ptr; // Mersenne twister engine
static std::uniform_real_distribution<>* ran_ptr; // Uniform distribution

static int thread; // Unique thread number
#pragma omp threadprivate(thread, gen_ptr, ran_ptr)

```

(a) Global definition of thread-private variables.

```cpp
// Generate random initial conditions for N masses
std::tuple<Vec, Vec> initial_conditions() {
    Vec x(ND), v(ND); // Allocate memory
    const double dx = x_max- x_min; // Position range
    const double dv = v_max- v_min; // Velocity range
    #pragma omp parallel for
    for (int i = 0; i < ND; ++i) {
        x[i] = (*ran_ptr)(*gen_ptr) * dx + x_min; // Random initial positions
        v[i] = (*ran_ptr)(*gen_ptr) * dv + v_min; // Random initial velocities
    }
    return {x, v}; // Positions and velocities
}

```

(b) OpenMP parallelized initial_conditions function.

```cpp
// Compute the acceleration of all masses
// a_i = G * sum_{j≠i} m_j * (x_j- x_i) / |x_j- x_i|^3
Vec acceleration(const Vec& x, const Vec& m) {
    Vec a(ND);// Accelerations
    #pragma omp parallel for
    for (int i = 0; i < N; ++i) {
        const int iD = i * D; // Flatten the index
        double dx[D]; // Difference in position
        for (int j = 0; j < N; ++j) {
            const int jD = j * D; // Flatten the index
            double dx2 = epsilon2; // Distance^2 (softened)
            for (int k = 0; k < D; ++k) {
                dx[k] = x[jD + k]- x[iD + k]; // Difference in position
                dx2 += dx[k] * dx[k]; // Distance^2
            }
            const double Gm_dx3 = G * m[j] / (dx2 * sqrt(dx2)); // G * m_j / |dx|^3
            for (int k = 0; k < D; ++k) {
                const int iDk = iD + k; // Flatten the index
                a[iDk] += Gm_dx3 * dx[k]; // Acceleration
            }
        }
    }
    return a; // Accelerations
}

```

(c) OpenMP parallelized acceleration function.

```cpp
// Compute the next position and velocity for all masses
std::tuple<Vec, Vec> timestep(const Vec& x0, const Vec& v0, const Vec& m) {
    Vec a0 = acceleration(x0, m); // Calculate particle accelerations
    Vec x1(ND), v1(ND); // Allocate memory
    #pragma omp parallel for
    for (int i = 0; i < ND; ++i) {
        v1[i] = a0[i] * dt + v0[i]; // New velocity
        x1[i] = v1[i] * dt + x0[i]; // New position
    }
    return {x1, v1}; // New positions and velocities
}

```

(d) OpenMP parallelized timestep function.

```cpp
// Main function
int main(int argc, char** argv) {
    // Start timing
    auto start = std::chrono::high_resolution_clock::now();

    #pragma omp parallel
    {
        thread = omp_get_thread_num(); // Unique thread number
        gen_ptr = new std::mt19937;
        (*gen_ptr).seed(thread); // Seed the random number generator
        ran_ptr = new std::uniform_real_distribution<>(0., 1.);
    }

```

(e) Thread-private allocation and seeding of RNGs.

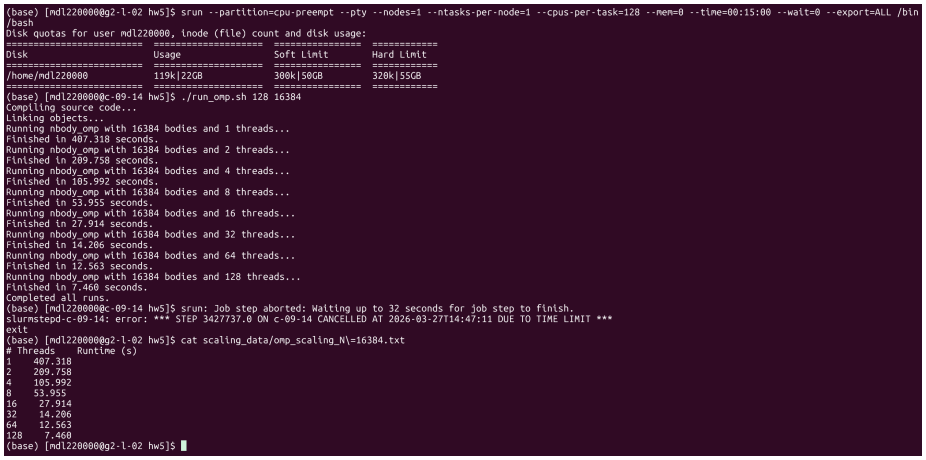
```cpp
    // Calculate the total kinetic energy of the system
    Vec KE(T+1); // Kinetic energy
    #pragma omp parallel for
    for (int n = 0; n <= T; ++n) {
        double KE_n = 0.; // Kinetic energy
        auto &v_n = v[n]; // Velocities
        for (int i = 0; i < N; ++i) {
            double v2 = 0.; // Velocity magnitude
            for (int j = 0; j < D; ++j) {
                const int k = i * D + j; // Flatten the index
                v2 += v_n[k] * v_n[k]; // Velocity magnitude
            }
            KE_n += 0.5 * m[i] * v2; // Kinetic energy
        }
        KE[n] = KE_n; // Kinetic energy
    }

    // Release memory
    #pragma omp parallel
    {
        delete gen_ptr;
        delete ran_ptr;
    }

```

(f) OpenMP parallelized kinetic energy calculations and memory release of thread-private RNGs.

Figure 2: Shared-memory OpenMP: key code snippets.



```
(base) [mdl220000@g2-l-02 hw5]$ srun --partition=cpu-preempt --pty --nodes=1 --ntasks-per-node=1 --cpus-per-task=128 --mem=0 --time=00:15:00 --wait=0 --export=ALL /bin
/bash
Disk quotas for user mdl220000, inode (file) count and disk usage:
=========================  ====================  ================  ============
Disk                       Usage                 Soft Limit        Hard Limit
=========================  ====================  ================  ============
/home/mdl220000            119k|22GB             300k|50GB         320k|55GB
=========================  ====================  ================  ============
(base) [mdl220000@c-09-14 hw5]$ ./run_omp.sh 128 16384
Compiling source code...
Linking objects...
Running nbody_omp with 16384 bodies and 1 threads...
Finished in 407.318 seconds.
Running nbody_omp with 16384 bodies and 2 threads...
Finished in 209.758 seconds.
Running nbody_omp with 16384 bodies and 4 threads...
Finished in 105.992 seconds.
Running nbody_omp with 16384 bodies and 8 threads...
Finished in 53.955 seconds.
Running nbody_omp with 16384 bodies and 16 threads...
Finished in 27.914 seconds.
Running nbody_omp with 16384 bodies and 32 threads...
Finished in 14.206 seconds.
Running nbody_omp with 16384 bodies and 64 threads...
Finished in 12.563 seconds.
Running nbody_omp with 16384 bodies and 128 threads...
Finished in 7.460 seconds.
Completed all runs.
(base) [mdl220000@c-09-14 hw5]$ srun: Job step aborted: Waiting up to 32 seconds for job step to finish.
slurmstepd-c-09-14: error: *** STEP 3427737.0 ON c-09-14 CANCELLED AT 2026-03-27T14:47:11 DUE TO TIME LIMIT ***
exit
(base) [mdl220000@g2-l-02 hw5]$ cat scaling_data/omp_scaling_N\=16384.txt
# Threads    Runtime (s)
1    407.318
2    209.758
4    105.992
8    53.955
16    27.914
32    14.206
64    12.563
128    7.460
(base) [mdl220000@g2-l-02 hw5]$
```

Figure 3: Ganymede2 terminal log for OpenMP scaling.

```cpp
static int rank, n_ranks; // Process rank and number of processes
static std::vector<int> counts, displs; // Counts and displacements for MPI_Allgatherv
static std::vector<int> countsD, displsD; // State counts and displacements for MPI_Allgatherv
static int N_beg, N_end, N_local; // Mass range for each process [N_beg, N_end)
static int ND_beg, ND_end, ND_local; // State vector range for each process [ND_beg, ND_end)

// Set up parallelism
void setup_parallelism() {
    MPI_Init(NULL, NULL); // Initialize MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Unique process rank
    MPI_Comm_size(MPI_COMM_WORLD, &n_ranks);

    // Get the current time and convert it to an integer
    auto now = std::chrono::high_resolution_clock::now();
    auto now_cast = std::chrono::time_point_cast<std::chrono::microseconds>(now);
    auto now_int = now_cast.time_since_epoch().count();

    // Pure MPI version
    gen.seed(now_int ^ rank); // Seed the random number generator

    // Divide the masses among the processes (needed for MPI_Allgatherv)
    counts.resize(n_ranks); // Counts for each process
    displs.resize(n_ranks); // Displacements for each process
    countsD.resize(n_ranks); // State counts for each process
    displsD.resize(n_ranks); // State displacements for each process
    const int remainder = N % n_ranks; // Remainder of the division
    for (int i = 0; i < n_ranks; ++i) {
        counts[i] = N / n_ranks; // Divide the masses among the processes
        displs[i] = i * counts[i]; // Displacements where each segment begins
        if (i < remainder) {
            counts[i] += 1; // Correct the count
            displs[i] += i; // Correct the displacement
        } else {
            displs[i] += remainder; // Correct the displacement
        }
        countsD[i] = counts[i] * D; // State counts for each process
        displsD[i] = displs[i] * D; // State displacements for each process
    }

    // Set up the local mass ranges
    N_beg = displs[rank]; // Mass range for each process [N_beg, N_end)
    N_end = N_beg + counts[rank]; // Mass range for each process [N_beg, N_end)
    ND_beg = N_beg * D; // State vector range for each process [ND_beg, ND_end)
    ND_end = N_end * D; // State vector range for each process [ND_beg, ND_end)
    N_local = N_end - N_beg; // Local number of masses
    ND_local = ND_end - ND_beg; // Local size of the state vectors
}
```

(a) Global variables to be distributed across MPI ranks.

```cpp
// Generate random initial conditions for N masses
std::tuple<Vec, Vec> initial_conditions() {
    Vec x(ND), v(ND); // Allocate memory
    const double dx = x_max - x_min; // Position range
    const double dv = v_max - v_min; // Velocity range
    for (int i = ND_beg; i < ND_end; ++i) {
        x[i] = ran(gen) * dx + x_min; // Random initial positions
        v[i] = ran(gen) * dv + v_min; // Random initial velocities
    }
    if (n_ranks > 1) { // More than one process
        // Gather the initial positions and velocities
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, x.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, v.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
    }
    return (x, v); // Positions and velocities
}

// Compute the acceleration of all masses
// a_i = G * sum_{j!=i} m_j * (x_j - x_i) / |x_j - x_i|^3
Vec acceleration(const Vec& x, const Vec& m) {
    Vec a(ND);// Accelerations
    for (int i = N_beg; i < N_end; ++i) {
        const int iD = i * D; // Flatten the index
        double dx[D]; // Difference in position
        for (int j = 0; j < N; ++j) {
            const int jD = j * D; // Flatten the index
            double dx2 = epsilon2; // Distance^2 (softened)
            for (int k = 0; k < D; ++k) {
                dx[k] = x[jD + k] - x[iD + k]; // Difference in position
                dx2 += dx[k] * dx[k]; // Distance^2
            }
            const double Gm_dx3 = G * m[j] / (dx2 * sqrt(dx2)); // G * m_j / |dx|^3
            for (int k = 0; k < D; ++k) {
                const int iDk = iD + k; // Flatten the index
                a[iDk] += Gm_dx3 * dx[k]; // Acceleration
            }
        }
    }
    return a; // Accelerations
}
```

(b) Initial conditions and acceleration local to MPI ranks.

```cpp
// Compute the next position and velocity for all masses
std::tuple<Vec, Vec> timestep(const Vec& x0, const Vec& v0, const Vec& m) {
    Vec a0 = acceleration(x0, m); // Calculate particle accelerations
    Vec x1(ND), v1(ND); // Allocate memory
    for (int i = ND_beg; i < ND_end; ++i) {
        v1[i] = a0[i] * dt + v0[i]; // New velocity
        x1[i] = v1[i] * dt + x0[i]; // New position
    }
    if (n_ranks > 1) { // More than one process
        // Gather the initial positions and velocities
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, x1.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, v1.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
    }
    return (x1, v1); // New positions and velocities
}

// Main function
int main(int argc, char** argv) {
    // Start timing
    auto start = std::chrono::high_resolution_clock::now();

    // Set up the problem
    if (argc > 3) {
        N = std::atoi(argv[3]); // Update the number of masses
        ND = N * D; // Update the size of the state vectors
    }

    setup_parallelism(); // Initialize MPI

    // Prepare vectors for time points, masses, positions, velocities, and kinetic energy
    Vec t(T+1); // Time points
    for (int i = 0; i <= T; ++i)
        t[i] = double(i) * dt; // Time points

    Vec m(N, m_0); // Masses (all equal)
    Vecs x(T+1), v(T+1); // Positions and velocities
    std::tie(x[0], v[0]) = initial_conditions(); // Set up initial conditions

    // Simulate the motion of N masses in D-dimensional space
    for (int n = 0; n < T; ++n)
        std::tie(x[n+1], v[n+1]) = timestep(x[n], v[n], m); // Time step
```

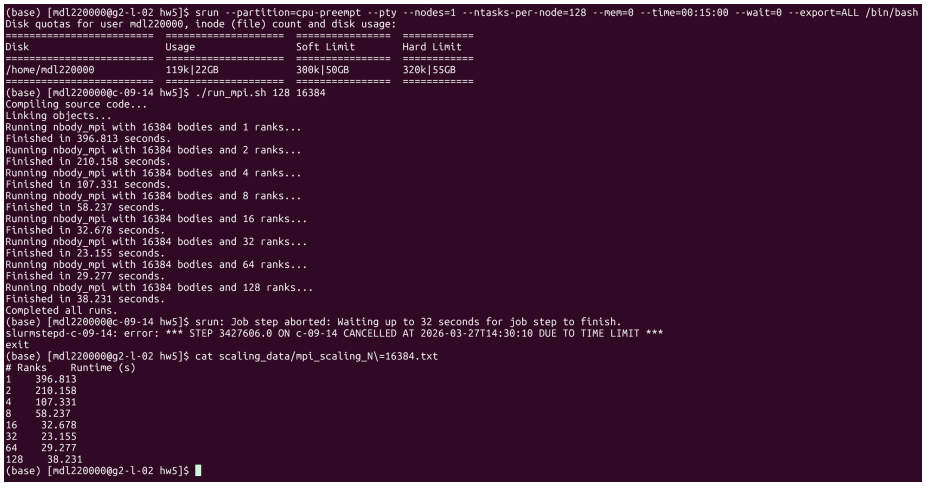
(c) MPI-localized timesteps and main function setup.

```cpp
    // Calculate the total kinetic energy of the system
    Vec KE(T+1); // Kinetic energy
    for (int n = 0; n <= T; ++n) {
        double KE_n = 0.; // Kinetic energy
        auto &v_n = v[n]; // Velocities
        for (int i = N_beg; i < N_end; ++i) {
            double v2 = 0.; // Velocity magnitude
            for (int j = 0; j < D; ++j) {
                const int k = i * D + j; // Flatten the index
                v2 += v_n[k] * v_n[k]; // Velocity magnitude
            }
            KE_n += 0.5 * m[i] * v2; // Kinetic energy
        }
        KE[n] = KE_n; // Kinetic energy
    }
    if (rank == 0) {
        // Reduce the kinetic energies
        MPI_Reduce(MPI_IN_PLACE, KE.data(), T+1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
        // Protected save and print logic ...
        save(KE, "nbody_output/KE_" + std::to_string(N) + ".txt", "Kinetic Energy");
        save(t, "nbody_output/time_" + std::to_string(N) + ".txt", "Time");
        /*
        // Output the results
        std::cout << "Total Kinetic Energy = [" << KE[0];
        const int T_skip = T / 50; // Skip every T_skip time steps
        for (int n = 1; n <= T; n += T_skip)
            std::cout << ", " << KE[n];
        std::cout << "]" << std::endl;
        */
    } else {
        // Send the kinetic energies
        MPI_Reduce(KE.data(), NULL, T+1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize(); // Finalize MPI
```

(d) Kinetic energy calculation and MPI finalization.

Figure 4: Distributed-memory MPI: key code snippets.



```
(base) [mdl220000@g2-l-02 hw5]$ srun --partition=cpu-preempt --pty --nodes=1 --ntasks-per-node=128 --mem=0 --time=00:15:00 --wait=0 --export=ALL /bin/bash
Disk quotas for user mdl220000, inode (file) count and disk usage:
========================  ====================  ================  ============
Disk                      Usage                 Soft Limit        Hard Limit
========================  ====================  ================  ============
/home/mdl220000           119k|22GB             300k|50GB         320k|55GB
========================  ====================  ================  ============
(base) [mdl220000@c-09-14 hw5]$ ./run_mpi.sh 128 16384
Compiling source code...
Linking objects...
Running nbody_mpi with 16384 bodies and 1 ranks...
Finished in 396.813 seconds.
Running nbody_mpi with 16384 bodies and 2 ranks...
Finished in 210.158 seconds.
Running nbody_mpi with 16384 bodies and 4 ranks...
Finished in 107.331 seconds.
Running nbody_mpi with 16384 bodies and 8 ranks...
Finished in 58.237 seconds.
Running nbody_mpi with 16384 bodies and 16 ranks...
Finished in 32.678 seconds.
Running nbody_mpi with 16384 bodies and 32 ranks...
Finished in 23.155 seconds.
Running nbody_mpi with 16384 bodies and 64 ranks...
Finished in 29.277 seconds.
Running nbody_mpi with 16384 bodies and 128 ranks...
Finished in 38.231 seconds.
Completed all runs.
(base) [mdl220000@c-09-14 hw5]$ srun: Job step aborted: Waiting up to 32 seconds for job step to finish.
slurmstepd-c-09-14: error: *** STEP 3427606.0 ON c-09-14 CANCELLED AT 2026-03-27T14:30:10 DUE TO TIME LIMIT ***
exit
(base) [mdl220000@g2-l-02 hw5]$ cat scaling_data/mpi_scaling_N\=16384.txt
# Ranks     Runtime (s)
1     396.813
2     210.158
4     107.331
8     58.237
16     32.678
32     23.155
64     29.277
128     38.231
(base) [mdl220000@g2-l-02 hw5]$
```

Figure 5: Ganymede2 terminal log for distributed-memory MPI scaling.

```cpp
    // Allocate shared memory for positions, velocities, and accelerations
    if (rank == 0) {
        MPI_Win_allocate_shared(N * sizeof(double), sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &m, &win_m);
        MPI_Win_allocate_shared(ND * sizeof(double), sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &x, &win_x);
        MPI_Win_allocate_shared(ND * sizeof(double), sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &v, &win_v);
        MPI_Win_allocate_shared(ND * sizeof(double), sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &a, &win_a);
        MPI_Win_allocate_shared(ND * sizeof(double), sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &x_next, &win_x_next);
        MPI_Win_allocate_shared(ND * sizeof(double), sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &v_next, &win_v_next);
    } else {
        MPI_Win_allocate_shared(0, sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &m, &win_m);
        MPI_Win_allocate_shared(0, sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &x, &win_x);
        MPI_Win_allocate_shared(0, sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &v, &win_v);
        MPI_Win_allocate_shared(0, sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &a, &win_a);
        MPI_Win_allocate_shared(0, sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &x_next, &win_x_next);
        MPI_Win_allocate_shared(0, sizeof(double), MPI_INFO_NULL, MPI_COMM_WORLD, &v_next, &win_v_next);
```

(a) Allocation of shared memory windows in MPI setup function.

```cpp
// Generate random initial conditions for N masses
void initial_conditions() {
    const double dx = x_max - x_min; // Position range
    const double dv = v_max - v_min; // Velocity range
    for (int i = ND_beg; i < ND_end; ++i) {
        x[i] = ran(gen) * dx + x_min; // Random initial positions
        v[i] = ran(gen) * dv + v_min; // Random initial velocities
    }
    MPI_Barrier(MPI_COMM_WORLD); // Synchronize all processes
    return;
}

// Compute the acceleration of all masses
// a_i = G * sum_{ji} m_j * (x_j - x_i) / |x_j - x_i|^3
void acceleration() {
    for (int i = N_beg; i < N_end; ++i) {
        const int iD = i * D; // Flatten the index
        double dx[D]; // Difference in position
        for (int j = 0; j < N; ++j) {
            const int jD = j * D; // Flatten the index
            double dx2 = epsilon2; // Distance^2 (softened)
            for (int k = 0; k < D; ++k) {
                dx[k] = x[jD + k] - x[iD + k]; // Difference in position
                dx2 += dx[k] * dx[k]; // Distance^2
            }
            const double Gm_dx3 = G * m[j] / (dx2 * sqrt(dx2)); // G * m_j / |dx|^3
            for (int k = 0; k < D; ++k) {
                const int iDk = iD + k; // Flatten the index
                a[iDk] += Gm_dx3 * dx[k]; // Acceleration
            }
        }
    }
    return;
}
```

(b) Initial conditions and acceleration calculation using shared memory between ranks.

```cpp
// Compute the next position and velocity for all masses
void timestep() {
    acceleration(); // Calculate particle accelerations
    for (int i = ND_beg; i < ND_end; ++i) {
        v_next[i] = a[i] * dt + v[i]; // New velocity
        x_next[i] = v_next[i] * dt + x[i]; // New position
    }
    MPI_Barrier(MPI_COMM_WORLD); // Synchronize all processes
    if (rank == 0) {
        std::swap(x, x_next); // Update the positions and velocities
        std::swap(v, v_next); // By swapping the pointers to the arrays
    }
    return;
}

double kinetic_energy() {
    // Calculate the total kinetic energy of the system
    double KE_n = 0.; // Kinetic energy
    for (int i = N_beg; i < N_end; ++i) {
        double v2 = 0.; // Velocity magnitude
        for (int j = 0; j < D; ++j) {
            const int k = i * D + j; // Flatten the index
            v2 += v[k] * v[k]; // Velocity magnitude
        }
        KE_n += 0.5 * m[i] * v2; // Kinetic energy
    }
    return KE_n; // Kinetic energy
}
```

(c) Timestep and kinetic energy calculations using shared memory between ranks.

```cpp
    setup_parallelism(); // Initialize MPI

    // Prepare vectors for time points, masses, positions, velocities, accelerations, and kinetic energy
    Vec t(T+1); // Time points
    for (int i = 0; i <= T; ++i)
        t[i] = double(i) * dt; // Time points

    m = (double *)malloc(N * sizeof(double)); // Masses (all equal)
    x = (double *)malloc(ND * sizeof(double)); // Positions
    v = (double *)malloc(ND * sizeof(double)); // Velocities
    a = (double *)malloc(ND * sizeof(double)); // Accelerations
    x_next = (double *)malloc(ND * sizeof(double)); // Position updates
    v_next = (double *)malloc(ND * sizeof(double)); // Velocity updates

    Vec KE(T+1); // Kinetic energy
    initial_conditions(); // Set up initial conditions
    KE[0] = kinetic_energy(); // Calculate initial kinetic energy

    // Simulate the motion of N masses in D-dimensional space
    for (int n = 0; n < T; ++n) {
        timestep(); // Time step
        KE[n] = kinetic_energy(); // Kinetic energy
    }
```

(d) Main function setup and calculations.

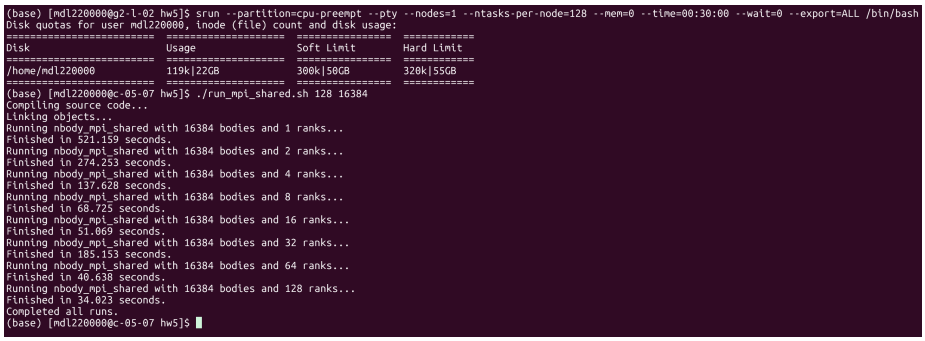
```cpp
    if (rank == 0) {
        // Reduce the kinetic energies
        MPI_Reduce(MPI_IN_PLACE, KE.data(), T+1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
        // Protected save and print logic ...
        save(KE, "nbody_output/KE_" + std::to_string(N) + ".txt", "Kinetic Energy");
        save(t, "nbody_output/time_" + std::to_string(N) + ".txt", "Time");
        /*
        // Output the results
        std::cout << "Total kinetic Energy = [" << KE[0];
        const int T_skip = T / 50; // Skip every T_skip time steps
        for (int n = 1; n <= T; n += T_skip)
            std::cout << ", " << KE[n];
        std::cout << "]" << std::endl;
        */
    } else {
        // Send the kinetic energies
        MPI_Reduce(KE.data(), NULL, T+1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    }

    // Free memory
    free(v_next);
    free(x_next);
    free(a);
    free(v);
    free(x);
    free(m);
    v_next = NULL;
    x_next = NULL;
    a = NULL;
    v = NULL;
    x = NULL;
    m = NULL;

    // Positions, velocities, and accelerations (reversed order)
    MPI_Win_free(&win_v_next);
    MPI_Win_free(&win_x_next);
    MPI_Win_free(&win_a);
    MPI_Win_free(&win_v);
    MPI_Win_free(&win_x);
    MPI_Win_free(&win_m);
    MPI_Finalize(); // Finalize MPI
```

(e) Kinetic energy reduction and freeing shared memory.

Figure 6: Shared-memory MPI: key code snippets.



```
(base) [mdl220000@g2-l-02 hw5]$ srun --partition=cpu-preempt --pty --nodes=1 --ntasks-per-node=128 --mem=0 --time=00:30:00 --wait=0 --export=ALL /bin/bash
Disk quotas for user mdl220000, inode (file) count and disk usage:
========================  ===================  ================  ============
Disk                      Usage                Soft Limit        Hard Limit
========================  ===================  ================  ============
/home/mdl220000           119k|22GB            300k|50GB         320k|55GB
========================  ===================  ================  ============
(base) [mdl220000@c-05-07 hw5]$ ./run_mpi_shared.sh 128 16384
Compiling source code...
Linking objects...
Running nbody_mpi_shared with 16384 bodies and 1 ranks...
Finished in 521.159 seconds.
Running nbody_mpi_shared with 16384 bodies and 2 ranks...
Finished in 274.253 seconds.
Running nbody_mpi_shared with 16384 bodies and 4 ranks...
Finished in 137.628 seconds.
Running nbody_mpi_shared with 16384 bodies and 8 ranks...
Finished in 68.725 seconds.
Running nbody_mpi_shared with 16384 bodies and 16 ranks...
Finished in 51.069 seconds.
Running nbody_mpi_shared with 16384 bodies and 32 ranks...
Finished in 185.153 seconds.
Running nbody_mpi_shared with 16384 bodies and 64 ranks...
Finished in 40.638 seconds.
Running nbody_mpi_shared with 16384 bodies and 128 ranks...
Finished in 34.023 seconds.
Completed all runs.
(base) [mdl220000@c-05-07 hw5]$
```

Figure 7: Ganymede2 terminal log for shared-memory MPI scaling.

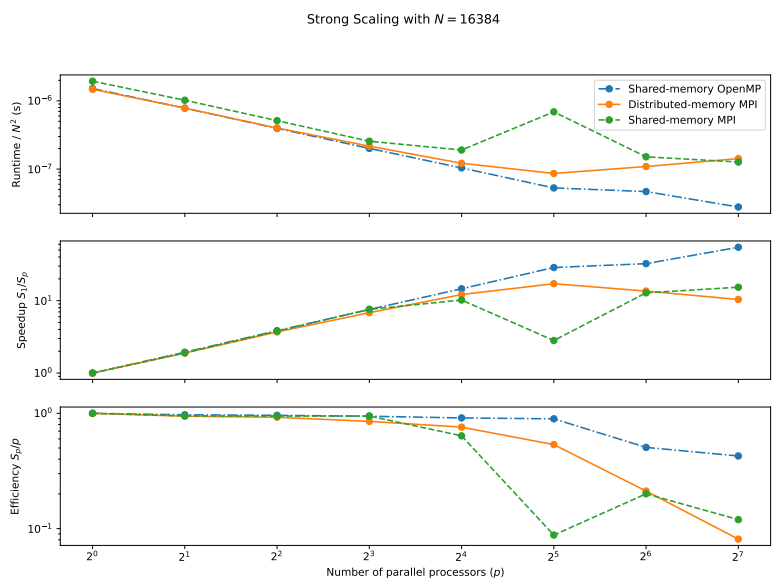


Figure 8: Strong scaling of OpenMP and MPI methods tested on a single node of Ganymede2. The scaling behaviors begin to diverge from one another at 16 parallel processors. The speedup of distributed-memory MPI tapers off, while the speedup of shared-memory MPI has an anomalous dip at 32 ranks.

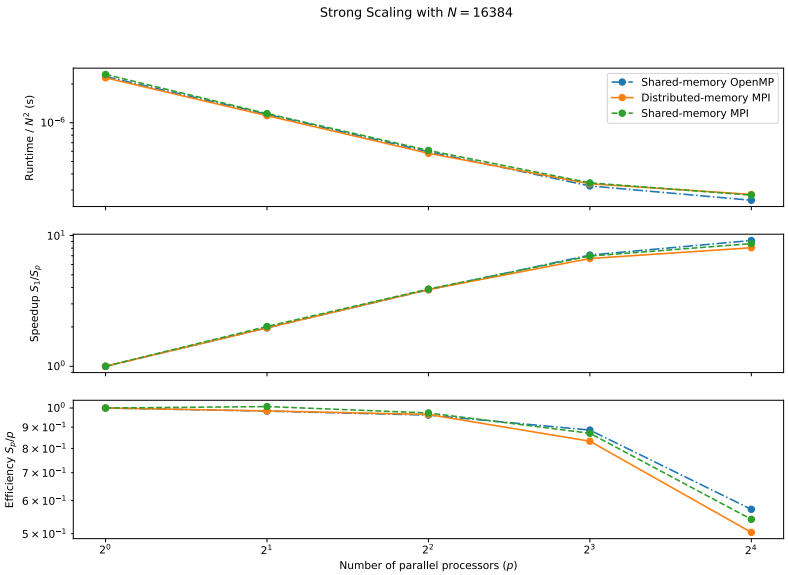


Figure 9: Strong scaling of OpenMP and MPI methods tested on my laptop. Qualitatively, this is in agreement with the scaling tests carried out on Ganymede2 for 16 and fewer parallel processors. At this scale, significant efficiency losses ($\sim 10\%$) are observed at 8 parallel processors.

```cpp
// Random number generator
static std::mt19937* gen_ptr; // Mersenne twister engine
static std::uniform_real_distribution<>* ran_ptr; // Uniform distribution

static int thread; // Unique thread number
#pragma omp threadprivate(thread, gen_ptr, ran_ptr)

static int rank, n_ranks; // Process rank and number of processes
static std::vector<int> counts, displs; // Counts and displacements for MPI_Allgatherv
static std::vector<int> countsD, displsD; // State counts and displacements for MPI_Allgatherv
static int N_beg, N_end, N_local; // Mass range for each process [N_beg, N_end)
static int ND_beg, ND_end, ND_local; // State vector range for each process [ND_beg, ND_end)
```

(a) Global variables used for OpenMP and MPI.

```cpp
void setup_parallelism() {
    MPI_Init(NULL, NULL); // Initialize MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Unique process rank
    MPI_Comm_size(MPI_COMM_WORLD, &n_ranks);

    // Get the current time and convert it to an integer
    auto now = std::chrono::high_resolution_clock::now();
    auto now_cast = std::chrono::time_point_cast<std::chrono::microseconds>(now);
    auto now_int = now_cast.time_since_epoch().count();

    // Hybrid MPI+OpenMP version
    #pragma omp parallel
    {
        thread = omp_get_thread_num(); // Unique thread number
        gen_ptr = new std::mt19937;
        (*gen_ptr).seed(now_int ^ (thread * n_ranks + rank)); // Seed the random number generator
        ran_ptr = new std::uniform_real_distribution<>(0., 1.);
    }

    // Divide the masses among the processes (needed for MPI_Allgatherv)
    counts.resize(n_ranks); // Counts for each process
    displs.resize(n_ranks); // Displacements for each process
    countsD.resize(n_ranks); // State counts for each process
    displsD.resize(n_ranks); // State displacements for each process
    const int remainder = N % n_ranks; // Remainder of the division
    for (int i = 0; i < n_ranks; ++i) {
        counts[i] = N / n_ranks; // Divide the masses among the processes
        displs[i] = i * counts[i]; // Displacements where each segment begins
        if (i < remainder) {
            counts[i] += 1; // Correct the count
            displs[i] += i; // Correct the displacement
        } else {
            displs[i] += remainder; // Correct the displacement
        }
        countsD[i] = counts[i] * D; // State counts for each process
        displsD[i] = displs[i] * D; // State displacements for each process
    }

    // Set up the local mass ranges
    N_beg = displs[rank]; // Mass range for each process [N_beg, N_end)
    N_end = N_beg + counts[rank]; // Mass range for each process [N_beg, N_end)
    ND_beg = N_beg * D; // State vector range for each process [ND_beg, ND_end)
    ND_end = N_end * D; // State vector range for each process [ND_beg, ND_end)
    N_local = N_end - N_beg; // Local number of masses
    ND_local = ND_end - ND_beg; // Local size of the state vectors
}
```

(b) Setup function for MPI, taking account of OpenMP threading.

```cpp
// Generate random initial conditions for N masses
std::tuple<Vec, Vec> initial_conditions() {
    Vec x(ND), v(ND); // Allocate memory
    const double dx = x_max - x_min; // Position range
    const double dv = v_max - v_min; // Velocity range
    #pragma omp parallel for
    for (int i = ND_beg; i < ND_end; ++i) {
        x[i] = (*ran_ptr)(*gen_ptr) * dx + x_min; // Random initial positions
        v[i] = (*ran_ptr)(*gen_ptr) * dv + v_min; // Random initial velocities
    }
    if (n_ranks > 1) { // More than one process
        // Gather the initial positions and velocities
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, x.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, v.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
    }
    return (x, v); // Positions and velocities
}

// Compute the acceleration of all masses
// a_i = G * sum_{j!=i} m_j * (x_j - x_i) / |x_j - x_i|^3
Vec acceleration(const Vec& x, const Vec& m) {
    Vec a(ND);// Accelerations
    #pragma omp parallel for
    for (int i = N_beg; i < N_end; ++i) {
        const int ID = i * D; // Flatten the index
        double dx[D]; // Difference in position
        for (int j = 0; j < N; ++j) {
            const int jD = j * D; // Flatten the index
            double dx2 = epsilon2; // Distance^2 (softened)
            for (int k = 0; k < D; ++k) {
                dx[k] = x[jD + k] - x[ID + k]; // Difference in position
                dx2 += dx[k] * dx[k]; // Distance^2
            }
            const double Gm_dx3 = G * m[j] / (dx2 * sqrt(dx2)); // G * m_j / |dx|^3
            for (int k = 0; k < D; ++k) {
                const int iDk = ID + k; // Flatten the index
                a[iDk] += Gm_dx3 * dx[k]; // Acceleration
            }
        }
    }
    return a; // Accelerations
}
```

(c) Initial conditions and acceleration for OpenMP + MPI.

```cpp
// Compute the next position and velocity for all masses
std::tuple<Vec, Vec> timestep(const Vec& x0, const Vec& v0, const Vec& m) {
    Vec a0 = acceleration(x0, m); // Calculate particle accelerations
    Vec x1(ND), v1(ND); // Allocate memory
    #pragma omp parallel for
    for (int i = ND_beg; i < ND_end; ++i) {
        v1[i] = a0[i] * dt + v0[i]; // New velocity
        x1[i] = v1[i] * dt + x0[i]; // New position
    }
    if (n_ranks > 1) { // More than one process
        // Gather the initial positions and velocities
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, x1.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
        MPI_Allgatherv(MPI_IN_PLACE, 0, MPI_DATATYPE_NULL, v1.data(), countsD.data(), displsD.data(), MPI_DOUBLE, MPI_COMM_WORLD);
    }
    return (x1, v1); // New positions and velocities
}

// Main function
int main(int argc, char** argv) {
    // Start timing
    auto start = std::chrono::high_resolution_clock::now();

    // Set up the problem
    if (argc > 1) {
        N = std::atoi(argv[1]); // Update the number of masses
        ND = N * D; // Update the size of the state vectors
    }

    setup_parallelism(); // Initialize MPI

    // Prepare vectors for time points, masses, positions, velocities, and kinetic energy
    Vec t(T+1); // Time points
    for (int i = 0; i <= T; ++i)
        t[i] = double(i) * dt; // Time points

    Vec m(N, m_0); // Masses (all equal)
    Vec x(T+1), v(T+1); // Positions and velocities
    std::tie(x[0], v[0]) = initial_conditions(); // Set up initial conditions

    // Simulate the motion of N masses in D-dimensional space
    for (int n = 0; n < T; ++n)
        std::tie(x[n+1], v[n+1]) = timestep(x[n], v[n], m); // Time step
```

(d) Main function setup and timesteps for OpenMP + MPI.

```cpp
    // Calculate the total kinetic energy of the system
    Vec KE(T+1); // Kinetic energy
    #pragma omp parallel for
    for (int n = 0; n <= T; ++n) {
        double KE_n = 0.; // Kinetic energy
        auto &v_n = v[n]; // Velocities
        for (int i = N_beg; i < N_end; ++i) {
            double v2 = 0.; // Velocity magnitude
            for (int j = 0; j < D; ++j) {
                const int k = i * D + j; // Flatten the index
                v2 += v_n[k] * v_n[k]; // Velocity magnitude
            }
            KE_n += 0.5 * m[i] * v2; // Kinetic energy
        }
        KE[n] = KE_n; // Kinetic energy
    }
    if (rank == 0) {
        // Reduce the kinetic energies
        MPI_Reduce(MPI_IN_PLACE, KE.data(), T+1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
        // Protected save and print logic ...
        save(KE, "nbody_output/KE_" + std::to_string(N) + ".txt", "Kinetic Energy");
        save(t, "nbody_output/time_" + std::to_string(N) + ".txt", "Time");
        /*
        // Output the results
        std::cout << "Total Kinetic Energy = [" << KE[0],
        const int T_skip = T / 50; // Skip every T_skip time steps
        for (int n = 1; n <= T; n += T_skip)
            std::cout << ", " << KE[n];
        std::cout << "]" << std::endl;
        */
    } else {
        // Send the kinetic energies
        MPI_Reduce(KE.data(), NULL, T+1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize(); // Finalize MPI
```

(e) Kinetic energy calculation and MPI finalization.

Figure 10: Hybrid MPI + OpenMP: key code snippets.

```
(base) [mdl220000@g2-l-02 hw5]$ sbatch run_mpi_omp.slurm 196608
Submitted batch job 3436877
(base) [mdl220000@g2-l-02 hw5]$ cat slurm-3436877.out
Compiling source code...
Linking objects...
Running nbody_mpi_omp with 196608 bodies, 4 ranks, and 32 threads...
Finished in 805.527 seconds.
(base) [mdl220000@g2-l-02 hw5]$ cat run_mpi_omp.slurm
#!/bin/bash
#SBATCH --job-name=mpi-omp
#SBATCH --partition=cpu-preempt
#SBATCH --nodes=2 # nodes
#SBATCH --ntasks-per-node=4 # total ranks
#SBATCH --cpus-per-task=32 # CPUs per rank
#SBATCH --time=00:15:00 # walltime
#SBATCH --exclusive # avoid interference
#SBATCH --mail-type=ALL
#SBATCH --mail-user=mdl220000@utdallas.edu

set -euo pipefail

N=${1:-16384}
n_ranks=${2:-4}
n_threads=${3:-32}

export OMP_NUM_THREADS=$n_threads

program="nbody_mpi_omp"
compiler="mpicxx -fopenmp -O3 -std=c++17"
echo "Compiling source code..."
$compiler -c $program.cc  # Compile the source file
echo "Linking objects..."
$compiler -o $program $program.o -lm # Link the object file
rm $program.o

TIMEFORMAT='%3R'
echo "Running $program with $N bodies, $n_ranks ranks, and $n_threads threads..."
real_time=$( { time mpiexec -n $n_ranks ./$program $N; } 2>&1 )
echo "Finished in $real_time seconds."
rm $program
(base) [mdl220000@g2-l-02 hw5]$
```

Figure 11: Ganymede2 terminal log for hybrid MPI + OpenMP on two nodes.